

Data Quality

Knowledge Objectives

1. Explain what data quality is
2. Exemplify the causes of data quality problems
3. Classify the data conflicts depending on:
 - a) They affect only the schema or also the instances
 - b) They can happen in a single data source or need many
4. Explain the different kinds of data quality rules in relational terms
5. Name the steps of object identification
6. Name and exemplify three kinds of space reduction
7. Name and exemplify four kinds of object identification functions

Understanding Objectives

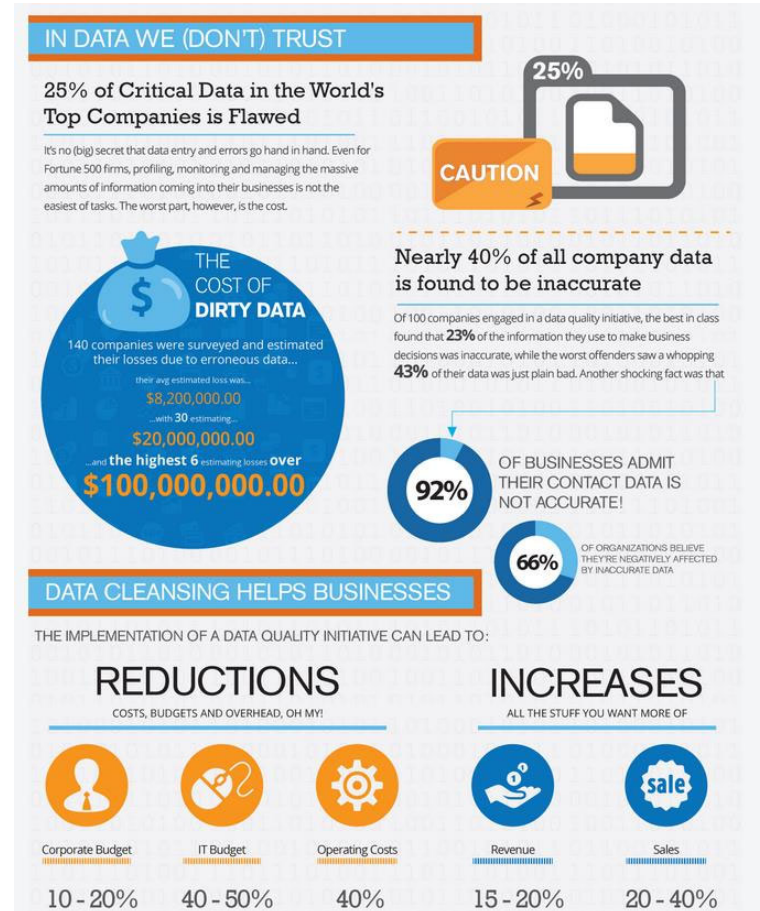
1. Calculate the value of the most prominent data quality measures (i.e., Completeness, Accuracy, Consistency, and Timeliness)

Application Objectives

1. Identify redundant rules
2. Identify inconsistent rules

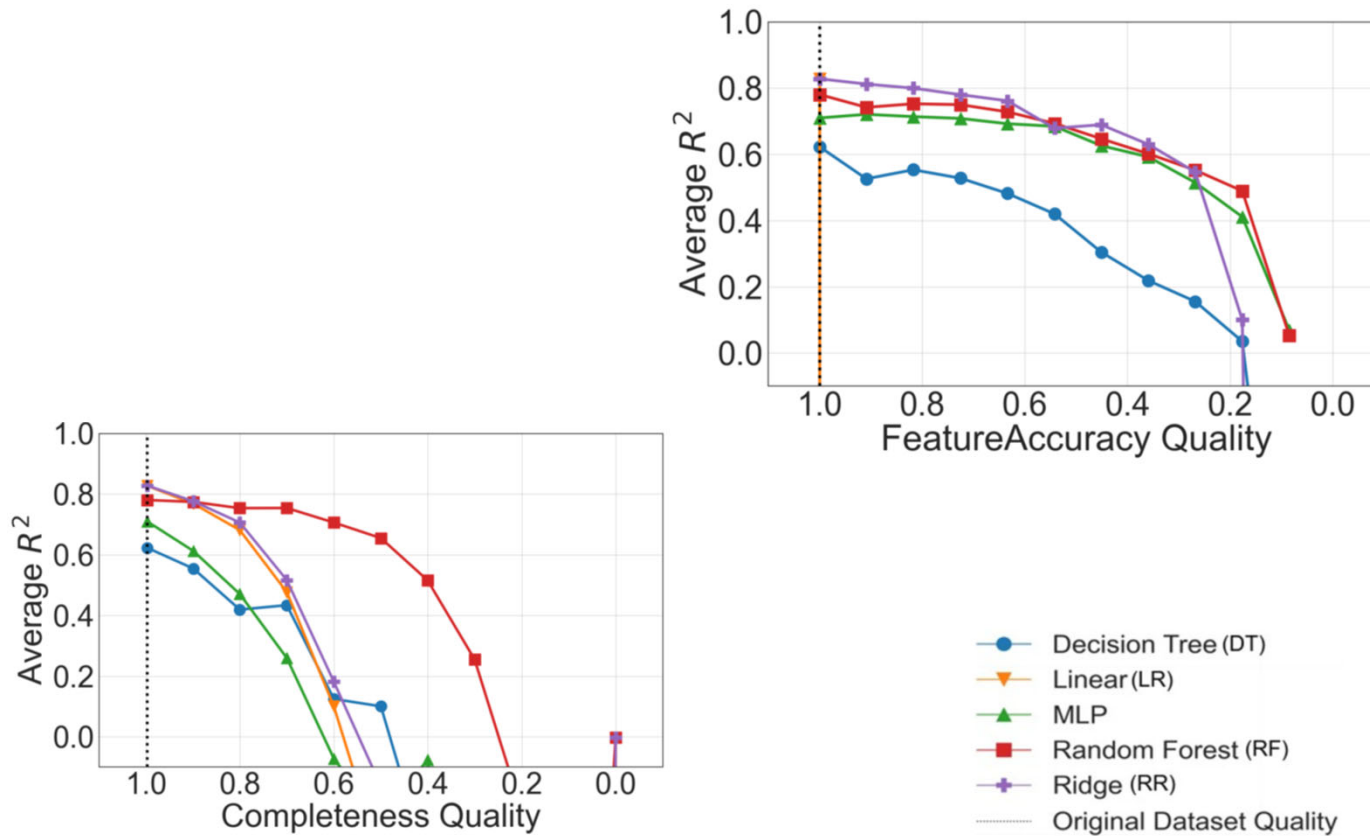
Motivation and Definition

Motivation (I)



Halo Business Intelligence

Motivation (II)



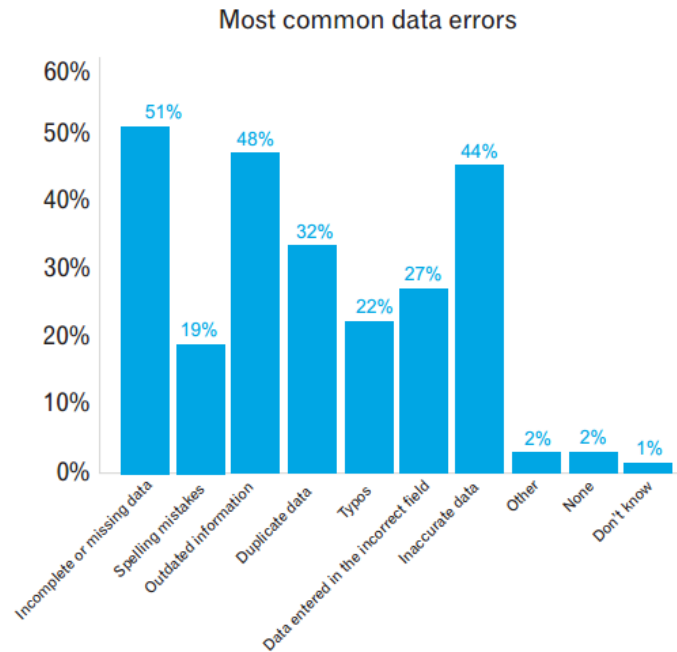
L. Budach, et al.

Fitness for use

“A user can only assess the level of quality of a set of data for a particular task to be executed in a **specific context**, according to a set of criteria, thus determining whether or not these data can be used **for that purpose**.”

William Edwards Deming

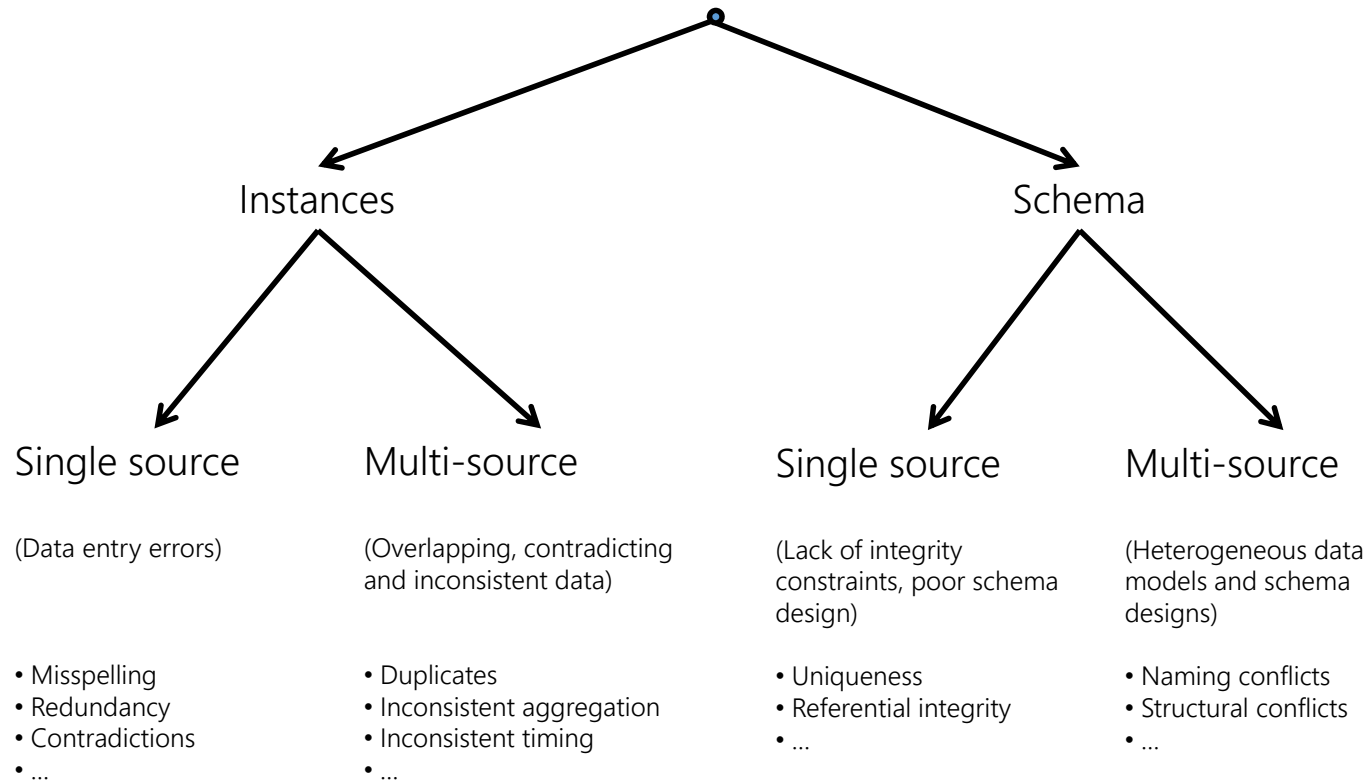
Most common data quality problems and impact



Classification of sources of problems

- Data ingestion
 - Initial data conversion
 - Manual data entry
 - Batch feeds
 - Real-time interfaces
 - System consolidations
- Data processing
 - Process automation
 - Data processing
 - Data cleansing
 - Data purging
- Inaction
 - Changes not captured
 - System upgrades
 - New data uses
 - Loss of expertise

Data conflicts



Quality Measures

Clusters of quality dimensions

- **Completeness, ...**
 - ...pertinence, and relevance refer to the capability of representing all and only the relevant aspects of the reality of interest
- **Accuracy, ...**
 - ... correctness, validity, and precision focus on the adherence to a given reality of interest (includes syntactic, semantic as well as **temporal** accuracy)
- **Consistency, ...**
 - ... cohesion, and coherence refer to the capability of the information to comply without contradictions to all properties of the reality of interest, as specified in terms of integrity constraints, data edits, business rules, and other formalisms
- **Redundancy, ...**
 - ... minimality, compactness, and conciseness refer to the capability of representing the aspects of the reality of interest with the minimal use of information resources
- **Readability, ...**
 - ... comprehensibility, clarity, and simplicity refer to easy understanding and fruition of information by users
- **Accessibility, ...**
 - ... and availability are related to the ability of the user to access information from his or her culture, physical status/functions, and technologies available
- **Usefulness, ...**
 - ... related to the advantage the user gains from the use of information
- **Trust, ...**
 - ... including believability, reliability, and reputation, catching how much information derives from an unauthoritative source (encompasses also issues related to security)

Completeness

“The degree to which a given collection of data describes the corresponding set of real-world objects.”

- Missing entities (OWA)
- Missing values (CWA)

$$Q_{Cm}(A_i) = |R(\text{NotNull}(A_i))|/|R|$$

$$Q_{Cm}(R) = |R(\bigwedge_{A_i \in R} \text{NotNull}(A_i))|/|R|$$

Accuracy

“The extent to which data are correct, reliable and certified error free.”

- Free of typing errors
- Appropriate precision

$$e_A = |v_A - v_{\text{RealWorld}}|$$

$$Q_A(A_i) = |R(e_{A_i} \leq \epsilon)| / |R|$$

$$Q_A(R) = |R(\bigwedge_{A_i \in R} e_{A_i} \leq \epsilon)| / |R|$$

Timeliness (Freshness)

“How old the stored value of an attribute is with regard to the current value in the real world.”

$$\text{age}(v) = \text{now} - \text{TransactionTime}$$

$$f_u(v) = \text{updates per time unit}$$

$$Q_T(v) = (1 + f_u(v) \cdot \text{age}(v))^{-1}$$

$$Q_T(A_i) = \text{Avg}_{v \in R[A_i]} Q_T(v)$$

$$Q_T(R) = \text{Avg}_{A_i \in R} Q_T(A_i)$$

Consistency

“The degree of violation of semantic rules defined over a set of data items.”

- Integrity constraints (a.k.a. business rules)
 - Entity
 - Domain
 - Referential
 - User-defined
- Coincidence of copies
 - Temporal
 - Permanent

$$Q_{Cn}(R,B) = |R(\bigwedge_{rule \in B} rule(A_1, \dots, A_n))|/|R|$$

Trade-offs between dimensions

- Timeliness $\Leftrightarrow$ Accuracy, Consistency and Completeness
- Completeness $\Leftrightarrow$ Accuracy and Consistency

Quality Rules

Integrity constraints in RDBMS

- Intra-Attribute
 - Domain
 - Outliers
 - Not null
- Intra-Tuple
 - Checks
- Intra-Relation
 - Primary keys
 - Unique
 - Temporal (Triggers)
 - State-dependent
- Inter-Relation
 - Foreign key
 - Assertions (Triggers)

Formalized as dependencies:

- Multivalued dependencies
 - Functional dependencies
$$t1.X=t2.X \Rightarrow t1.Y=t2.Y$$
 - Key dependencies
- Inclusion dependencies
$$R.X \subseteq S.Y$$
 - Foreign key dependencies

Problems in the rules

- Contradictory rules prevent insertions (can generate empty results)

CHECK ($a < 10$ AND $a > 20$)

- Redundant rules slow down performance

1. CHECK ($a > 20$)

2. CHECK ($a > 10$)

- Imperfections

Logic properties of rules

- *Schema-satisfiability*: A **schema** is satisfiable if there is at least one consistent DB state containing tuples (i.e. each and every constraint is fulfilled)
- *Liveliness*: A **table/view** is lively if there is at least one consistent DB state, so that the table/view contains tuples
- *Redundancy*: An **integrity constraint** is redundant if the consistency of the DB does not depend on it (none of the tuples it tries to avoid can never exist)
- *State-reachability*: A given **set of tuples** is reachable if there is at least one consistent DB state containing those tuples (and maybe others)
- *Query containment (subsumption)*: A **query** Q1 is contained in another Q2, if the set of tuples of Q1 is always contained in the set of tuples of Q2 in every consistent DB state

Example of Schema-satisfiability

```
CREATE TABLE employees (  
  id CHAR(9) PRIMARY KEY,  
  dpt VARCHAR(4) NOT NULL  
);
```

```
CREATE TABLE departments (  
  id VARCHAR(4) PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  basicSalary INT  
  ,  
  CONSTRAINT ckMinSalary CHECK (basicSalary>2000),  
  CONSTRAINT ckMaxSalary CHECK (basicSalary<1000)  
);
```

Example of Liveliness

```
CREATE TABLE departments (  
  id VARCHAR(4) PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  basicSalary INT NOT NULL,  
  CONSTRAINT ckMinSalary CHECK (basicSalary>2000) );
```

```
CREATE TABLE employees (  
  id CHAR(9) PRIMARY KEY,  
  dpt VARCHAR(4) REFERENCES departments (id) );
```

```
CREATE VIEW unassigned AS (  
  SELECT *  
  FROM employees e  
  WHERE NOT EXISTS (  
    SELECT *  
    FROM departments d  
    WHERE d.id=e.dpt) );
```

Example of Redundancy

```
CREATE TABLE departments (  
  id VARCHAR(4) PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  basicSalary INT NOT NULL,  
  CONSTRAINT ckMinSalary CHECK (basicSalary>2000),  
  CONSTRAINT ckDeptName CHECK (id<>'CS')  
);
```

```
CREATE TABLE employees (  
  id CHAR(9) PRIMARY KEY,  
  dpt VARCHAR(4) NOT NULL REFERENCES departments (ID),  
  CONSTRAINT ckEmpName CHECK (dpt<>'CS')  
);
```

Example of State-reachability

```
CREATE TABLE departments (  
  id VARCHAR(4) PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  basicSalary INT NOT NULL,  
  CONSTRAINT ckMinSalary CHECK (basicSalary > 2000));
```

```
CREATE TABLE employees (  
  id CHAR(9) PRIMARY KEY,  
  dpt VARCHAR(4) NOT NULL REFERENCES departments (ID));
```

Employees(id, dpt);	Departments(id,	name,	basicSalary)
1 CS	CS	Compu...	10000
2 MK			

Example of Query Containment (I)

A: SELECT *
 FROM departments
 WHERE basicSalary > 5000;

$$A \subseteq B$$

B: SELECT *
 FROM departments
 WHERE basicSalary > 6000;

Example of Query Containment (II)

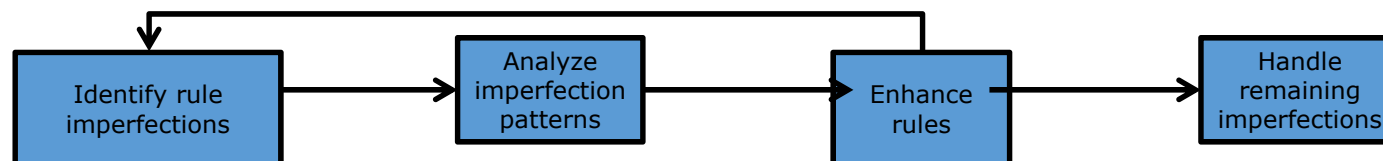
A: SELECT *
 FROM departments
 WHERE basicSalary > 5000;

$$A \subseteq B$$

B: SELECT *
 FROM departments
 WHERE basicSalary > 5000 AND name > 'M';

Fine-tuning imperfections

- Improvement management
 - False positives (correct tuples that incorrectly trigger the rule)
 - Since the rule is exceptionally triggered, FP should be very few
 - False negatives (incorrect tuples that the rule fails to detect)
 - If the number is relevant, FN should pop up in a relatively small sample
- Improvement monitoring



Quality Improvement

Object Identification

Quality improvement

a) Imputation

- Minimum change
- Keep marginal and joint frequency distribution of values in the different attributes

b) Acquisition of new data

1. Error localization
2. Source identification
 - Source trustworthiness
 - Cost optimization
3. Entity resolution/Record matching
 - Standardization/Normalization
4. Record merging

Object identification example

FIFA				
ID	Name	Position	Club	Year
158023	Lionel Andrés Messi Cuccittini	RW, ST, CF	Football Club Barcelona	2021

FBref				
ID	Name	Position	Club	Year
d70ce98e	L. Messi	Attack, Right Winger	Fc Paris Saint Germain	2021

TransferMarkt				
ID	Name	Position	Club	Year
28003	Lionel Messi	FW-MF	Paris Saint-Germain	2022

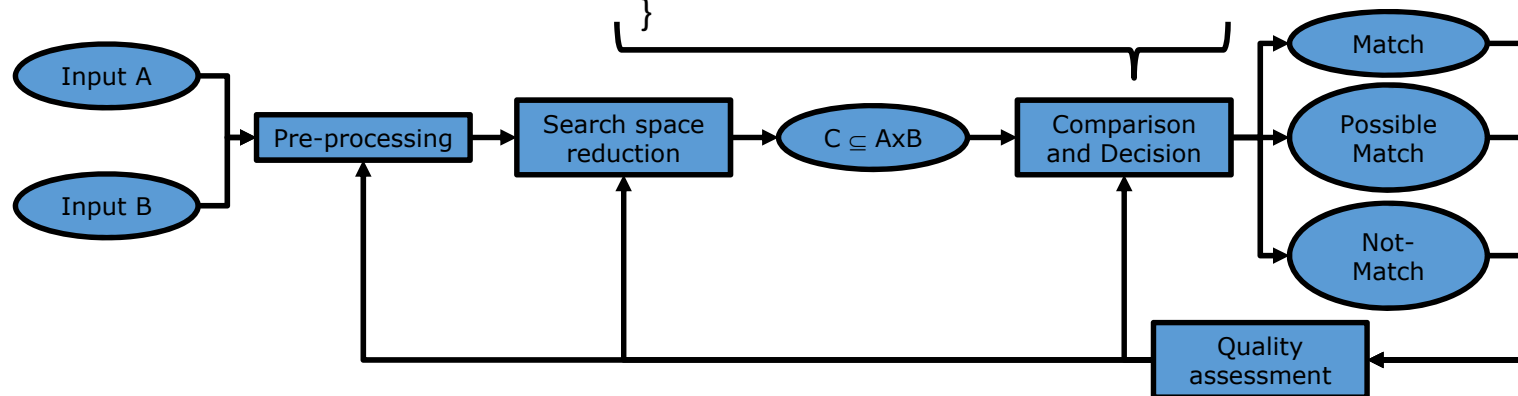
<https://www.kaggle.com/datasets/stefanoleone992/fifa-22-complete-player-dataset>

https://www.kaggle.com/datasets/goaltergeist/tidy-fbref-2021-player-data-credit-ronanmann?select=fb-data_2020-21_players_share.csv

<https://www.kaggle.com/datasets/davidcariboo/player-scores>

Process for Object identification

```
O := ∅;  
while (I <> ∅) do {  
  pick up r ∈ I; I := I - r;  
  if (∃ s ∈ O so that s ≈ r) {  
    O := O - s; I := I ∪ {r ∧ s};  
  } else {  
    O := O ∪ {r};  
  }  
}
```



C. Batini and M. Scannapieco

Search space reduction

- Blocking
 - Choose a key to partition the sets
- Sorted neighbourhood
 - Define a window to probe only records in it
- Pruning
 - Find some rules (heuristics)

Comparison/Similarity function ($\approx$)

- Distance-based
 - Decide that two records represent the same entity if their distance is below a threshold (after removing affixes)
 - Hamming distance
 - Edit distance (insert, delete and replace)
 - Smith-Waterman algorithm (assign weights)
 - n-Grams comparison
 - Jaro algorithm (insert, delete and transpose)
 - Compare lists of items
 - Jaccard distance
 - Token Frequency-Inverse Document Frequency (TF-IDF)
 - May use phonetics
 - Soundex code
- Rule-based
 - Depending on the concrete attributes (e.g., ignoring middle name)
- Probabilistic techniques (which require a threshold)
 - Based on conditional probabilities
- Classification-based
 - Provide positive and negative training pairs of instances (to train a classifier)

Closing

Summary

- Quality measures:
 - Accuracy
 - Completeness
 - Consistency
 - Timeliness
- Quality rules
 - Dependencies
 - Relational constraints
 - Consistency checks
 - Redundancy check
 - Imputation
- Object identification

References

- A. Maydanchik. *Data Quality Assessment*. Technics Publications, 2007
- J. Bleiholder and F. Naumann. Data Fusion. *ACM Computing Surveys*, 41(1), 2008
- C. Batini, et al. *Methodologies for data quality assessment and improvement*. *ACM Computing Surveys*, 41(3), 2009
- H. Garcia-Molina et al. *Database Systems*. Prentice Hall, 2009
- C. Batini and M. Scannapieco. *Data and Information Quality*. Springer, 2016
- L. Budach, et al. The effects of Data Quality on ML-Model Performance. *VLDB*, 2023